

```
// FTC practice code - Nikhil
// been teaching myself java + ftc stuff the past few weeks in ftcsim, python background before this
// these two show the main things i actually struggled with and fixed myself:
// 1) getting the robot to move a set distance properly instead of just guessing with sleep()
// 2) a colour detection bug that kept making the robot spin/twitch, and how i fixed it
```

```
// -----
// FILE 1 - EncoderAutonomous
// -----
// at first i was just doing motorLeft.setPower(1) then sleep(500) to move the robot,
// but it was so inconsistent - same code, different result every run (battery, floor grip etc)
// switched to using the encoders instead so it actually drives to a real tick count and knows
// when it's arrived, rather than just guessing a time and hoping
```

```
public class EncoderAutonomous extends LinearOpMode {
    DcMotor motorLeft;
    DcMotor motorRight;
    DcMotor frontLeft;
    DcMotor frontRight;

    @Override
    public void runOpMode() {
        motorLeft = hardwareMap.get(DcMotor.class, "motorLeft");
        motorRight = hardwareMap.get(DcMotor.class, "motorRight");
        frontLeft = hardwareMap.get(DcMotor.class, "frontLeft");
        frontRight = hardwareMap.get(DcMotor.class, "frontRight");

        motorLeft.setDirection(DcMotor.Direction.REVERSE);
        motorLeft.setMode(DcMotor.RunMode.STOP_AND_RESET_ENCODER);
        motorRight.setMode(DcMotor.RunMode.STOP_AND_RESET_ENCODER);

        waitForStart();

        moveTo(1000, 1000, 0.5);    // straight
        moveTo(-500, 500, 0.5);    // pivot turn, left back right forward
        moveTo(1500, 1500, 0.5);    // straight again

        motorLeft.setPower(0);
        motorRight.setPower(0);
    }

    // made this so i'm not copy pasting the same setTargetPosition/setMode/setPower
    // block every single time i want to move somewhere. just feed it ticks + power
    private void moveTo(int leftTicks, int rightTicks, double power) {
        int leftTarget = motorLeft.getCurrentPosition() + leftTicks;
        int rightTarget = motorRight.getCurrentPosition() + rightTicks;

        motorLeft.setTargetPosition(leftTarget);
        motorRight.setTargetPosition(rightTarget);

        motorLeft.setMode(DcMotor.RunMode.RUN_TO_POSITION);
        motorRight.setMode(DcMotor.RunMode.RUN_TO_POSITION);

        motorLeft.setPower(power);
        motorRight.setPower(power);

        // just wait here till both wheels actually reach where they're meant to be
        while (opModeIsActive() && motorLeft.isBusy() && motorRight.isBusy()) {
            telemetry.addData("Left ticks", motorLeft.getCurrentPosition());
            telemetry.addData("Right ticks", motorRight.getCurrentPosition());
            telemetry.update();
        }
    }
}
```

```

        motorLeft.setPower(0);
        motorRight.setPower(0);
    }
}

// -----
// FILE 2 - ColourDetectionAutonomous
// -----
// this one had a proper bug that took me a while to figure out - the robot would detect
// red, do its turn/reverse thing, but then just immediately detect red again straight after
// and do the whole thing again, over and over, basically spinning on the spot forever.
// turns out it was because the loop just goes straight back to checking colour again after
// the manoeuvre, and if the robot hasn't actually moved away from the panel yet it just
// re-triggers instantly.
// fixed it two ways:
// - added handledRed/handledBlue booleans so each one only ever fires once
// - swapped a single distance check for a proper loop that waits till it's ACTUALLY
// clear (under/over a threshold with a gap between them) instead of one snapshot reading

public class ColourDetectionAutonomous extends LinearOpMode {
    DcMotor motorLeft;
    DcMotor motorRight;
    ColorSensor color1;
    DistanceSensor distance1;

    @Override
    public void runOpMode() {
        motorLeft = hardwareMap.get(DcMotor.class, "motorLeft");
        motorRight = hardwareMap.get(DcMotor.class, "motorRight");
        color1 = hardwareMap.get(ColorSensor.class, "color1");
        distance1 = hardwareMap.get(DistanceSensor.class, "distance1");

        motorLeft.setDirection(DcMotor.Direction.REVERSE);

        waitForStart();

        // drive forward till we're near the colour panel
        while (opModeIsActive() && distance1.getDistance(DistanceUnit.CM) > 40) {
            motorLeft.setPower(0.5);
            motorRight.setPower(0.5);
            telemetry.addData("Distance", distance1.getDistance(DistanceUnit.CM));
            telemetry.update();
        }
        motorLeft.setPower(0);
        motorRight.setPower(0);

        boolean handledRed = false;
        boolean handledBlue = false;

        while (opModeIsActive()) {
            telemetry.addData("Red Value", color1.red());
            telemetry.addData("Blue Value", color1.blue());
            telemetry.update();

            // !handledRed is the actual fix - without this it just kept re-triggering
            if (color1.red() > 220 && !handledRed) {
                handledRed = true;
                motorLeft.setPower(-1);
                motorRight.setPower(1);
                sleep(420);
                motorLeft.setPower(0);
                motorRight.setPower(0);
            }
        }
    }
}

```

```

        // keep backing off until we're properly clear, not just a one time check
        while (opModeIsActive() && distance1.getDistance(DistanceUnit.CM) < 60) {
            motorLeft.setPower(-1);
            motorRight.setPower(-1);
            telemetry.addData("Distance", distance1.getDistance(DistanceUnit.CM));
            telemetry.update();
        }
        motorLeft.setPower(1);
        motorRight.setPower(1);
    }

    if (color1.blue() > 220 && !handledBlue) {
        handledBlue = true;
        motorLeft.setPower(-1);
        motorRight.setPower(1);
        sleep(420);
        motorLeft.setPower(0);
        motorRight.setPower(0);

        while (opModeIsActive() && distance1.getDistance(DistanceUnit.CM) < 60) {
            motorLeft.setPower(1);
            motorRight.setPower(-1);
            telemetry.addData("Distance", distance1.getDistance(DistanceUnit.CM));
            telemetry.update();
        }
        motorLeft.setPower(1);
        motorRight.setPower(1);
    }
}
}
}
}
}

```

```

// -----
// FILE 3 - DecodeAutonomous
// -----

```

```

// this one is meant to be a simplified version of what actual DECODE autonomous
// code does (DECODE is the 25/26 ftc season game). in the real game, robots have
// 30 seconds to sense the "motif" (a randomised colour pattern shown at the start
// of the match) then score artifacts (purple/green balls) into their goal, sorting
// them to match that pattern.
// i obviously can't build the real thing without the real field/game elements, but
// this mimics the actual logic flow: drive to the scoring area, read a colour sensor
// to "classify" what it's found (standing in for an artifact's colour), branch based
// on that, then return to a base zone and stop - using the same building blocks
// (encoder driving + colour/distance sensing) as my other two files

```

```

public class DecodeAutonomous extends LinearOpMode {
    DcMotor motorLeft;
    DcMotor motorRight;
    ColorSensor color1;
    DistanceSensor distance1;

    @Override
    public void runOpMode() {
        motorLeft = hardwareMap.get(DcMotor.class, "motorLeft");
        motorRight = hardwareMap.get(DcMotor.class, "motorRight");
        color1 = hardwareMap.get(ColorSensor.class, "color1");
        distance1 = hardwareMap.get(DistanceSensor.class, "distance1");

        motorLeft.setDirection(DcMotor.Direction.REVERSE);
        motorLeft.setMode(DcMotor.RunMode.STOP_AND_RESET_ENCODER);
        motorRight.setMode(DcMotor.RunMode.STOP_AND_RESET_ENCODER);
    }
}

```

```

waitForStart();

// drive to the "goal" - standing in for approaching the scoring area
moveTo(1200, 1200, 0.5);

// read the sensor once we're there and classify what we found
// (purple/green in the real game, using red/blue thresholds here since
// that's what i've got set up and tested in ftcsim)
telemetry.addData("Red Value", color1.red());
telemetry.addData("Blue Value", color1.blue());
telemetry.update();

if (color1.red() > 220) {
    // "classified" one way - go to scoring position A
    moveTo(300, 300, 0.4);
} else if (color1.blue() > 220) {
    // "classified" the other way - go to scoring position B
    moveTo(-300, -300, 0.4);
}

// gotta switch back out of RUN_TO_POSITION before using setPower() directly
// again here - learned this the hard way, motors ignore setPower() properly
// if you're still in RUN_TO_POSITION mode from the last moveTo() call
motorLeft.setMode(DcMotor.RunMode.RUN_USING_ENCODER);
motorRight.setMode(DcMotor.RunMode.RUN_USING_ENCODER);

// head back to base and stop once close, same pattern as my wall
// avoidance logic - keeps checking distance rather than guessing a time
while (opModeIsActive() && distance1.getDistance(DistanceUnit.CM) > 30) {
    motorLeft.setPower(-0.5);
    motorRight.setPower(-0.5);
    telemetry.addData("Distance to base", distance1.getDistance(DistanceUnit.CM));
    telemetry.update();
}
motorLeft.setPower(0);
motorRight.setPower(0);
}

private void moveTo(int leftTicks, int rightTicks, double power) {
    int leftTarget = motorLeft.getCurrentPosition() + leftTicks;
    int rightTarget = motorRight.getCurrentPosition() + rightTicks;

    motorLeft.setTargetPosition(leftTarget);
    motorRight.setTargetPosition(rightTarget);

    motorLeft.setMode(DcMotor.RunMode.RUN_TO_POSITION);
    motorRight.setMode(DcMotor.RunMode.RUN_TO_POSITION);

    motorLeft.setPower(power);
    motorRight.setPower(power);

    while (opModeIsActive() && motorLeft.isBusy() && motorRight.isBusy()) {
        telemetry.addData("Left ticks", motorLeft.getCurrentPosition());
        telemetry.addData("Right ticks", motorRight.getCurrentPosition());
        telemetry.update();
    }

    motorLeft.setPower(0);
    motorRight.setPower(0);
}
}

```